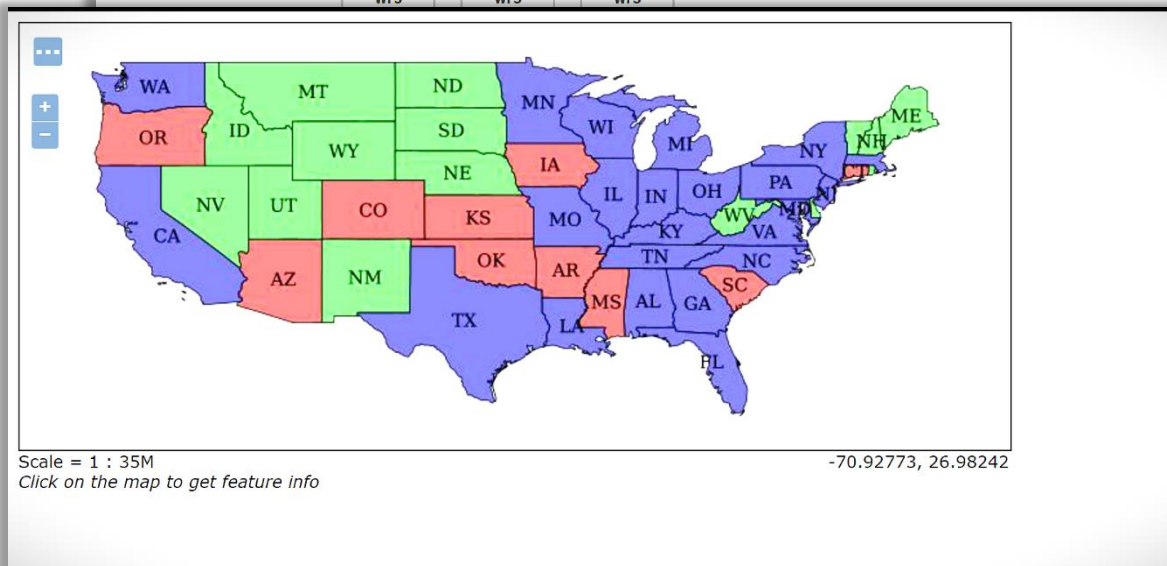
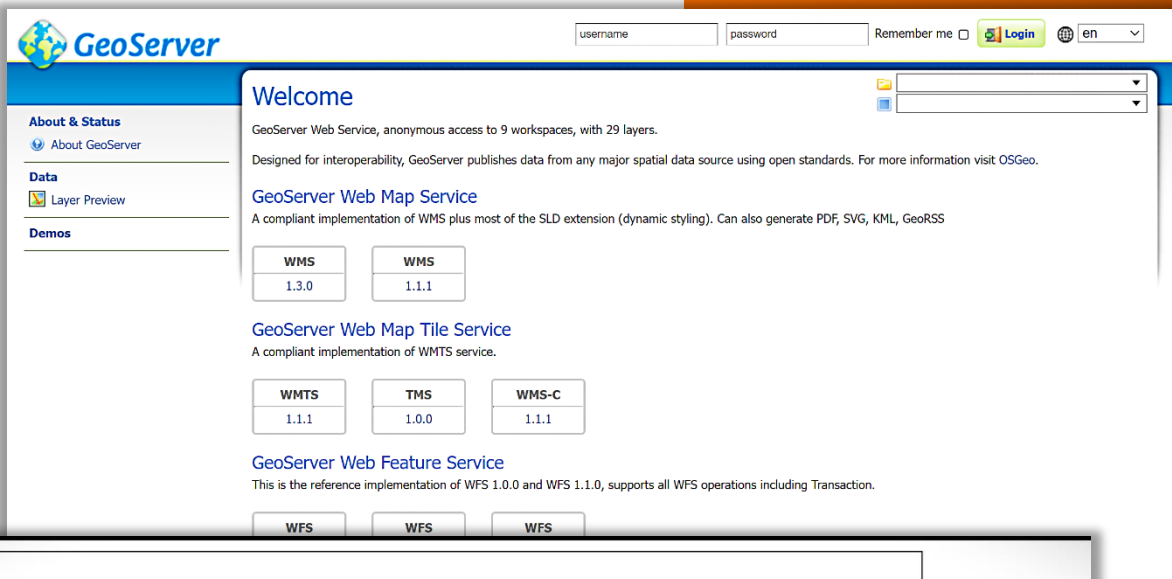




Praneeth Bhandwalkar

LinkedIn | GitHub

SERVERLESS GEOSERVER INFRASTRUCTURE



AWS Services Used:

- AWS Fargate
- Amazon RDS
- Amazon EFS
- Amazon VPC
- Amazon ECS

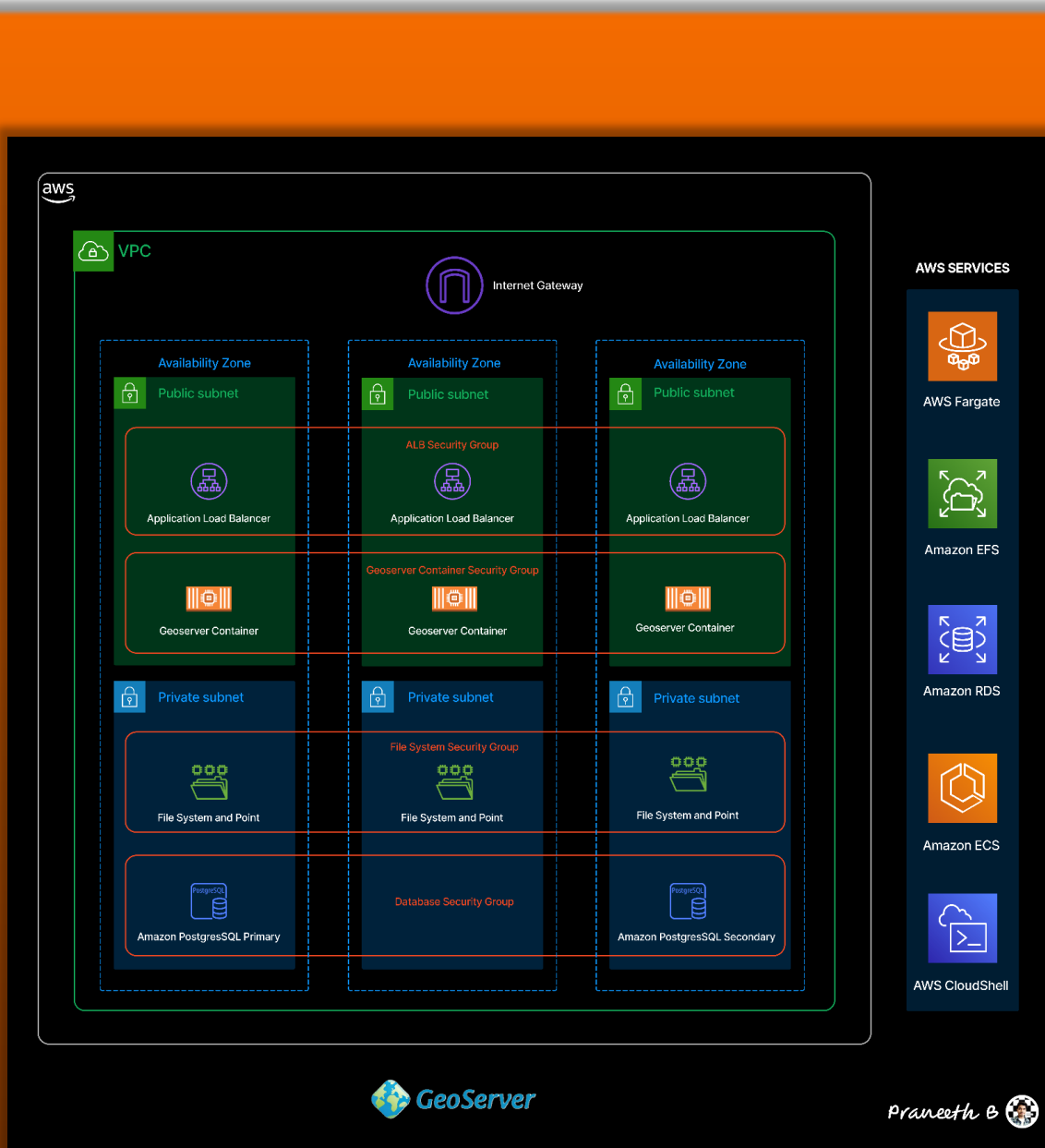
AWS Serverless GeoServer Infrastructure with PostgreSQL and PostGIS Extension

Description:

Deploy and auto-scale GeoServer with PostgreSQL and the PostGIS extension on AWS. This project utilizes containerized deployment through AWS Fargate, allowing you to concentrate on managing geospatial data without the burdens of server maintenance.

Architecture Overview:

The architecture is illustrated in the figure below.



HOW TO GET STARTED: Key Features:

PART - 1 - Set up infrastructure

- A Virtual Private Cloud (VPC) and create Security Groups.
- A PostgreSQL database using Amazon RDS.
- An Amazon Elastic File System (EFS) for shared storage.
- An administration server to allow you to perform database setup tasks.
- Security groups that will control access to the database and shared storage.

PART - 2 - Configure Geoserver

- Configure the PostgreSQL database including installing PostGIS.
- Populate the database with some geospatial data.
- Provision the GeoServer container.
- Create a load-balanced service for the Geoserver container.

PART - 1: Set up infrastructure

Step 1: Set up a Virtual Private Cloud (VPC)

Lets create a fresh VPC

1. Open the VPC console by typing **VPC** in the search bar at the top of the screen. Click the **VPC** title in the search results.
2. When the VPC dashboard loads click on **Create VPC**.
3. On the Create VPC screen do the following:
 - Select **VPC and more** in the Resources to Create section
 - Enter **geoserver** for Name tag generation
 - Select **3** for number of availability zones
 - Select **3** for number of public and private subnets
 - Select **1** for NAT gateways and **none** VPC endpoints.
4. Click **Create VPC**.

This will create and configure the VPC, subnets, route tables and Internet gateway. Once completed click the **View VPC** button to view what has been set up.

Note down the **VPC ID** of the VPC you have created as you will need this in the next step when you create the Security groups.

Step 2: Create security groups

Application Load Balancer Security Group

In the latter part of this lab, we will use an Application Load Balancer (ALB) to front end our Geoserver service. We will create the security group for this now so we can reference to it in another security group.

1. From the VPC console select Security Groups from the left hand menu. Once opened click the Create security group button in the top right hand corner.
2. Enter the following details:
 - Security group Name : HOLALB
 - Description : HOLALB
 - VPC : Click the dropdown in the VPC field then select the geoserver-vpc VPC.
3. In Inbound rules click Add rule
 - Select HTTP from the type drop down. This will select port 80
 - Select anywhere ipv4 0.0.0.0/0 from the Source field.
4. Click the Create security group button at bottom.

Similarly create security groups for other three with different inbound rules

- **HOLGeoserver security group** - Type: Custom TCP Source: MyIP and Add another rule with TCP but Source: HOLALB security group.
- **HOLEFS security group** - Type: NFS Source: HOLgeoserver security group.
- **HOLPostGIS security group** - Type: PostgreSQL Source: HOLgeoserver security group.

Step 3: Create Amazon Elastic File Server (EFS)

1. Type EFS into the search bar at the top of the screen. Click the EFS search result to open the EFS console.
2. Click the Create File System button
3. On the Create File System prompt enter:
 - Name: HOLEFS
 - Virtual Private Cloud: select the Geoserver-vpc that you created.
4. Click the Customize button and then scroll to the bottom of the page and click

Next.

5. In the Network Access screen select the HOLEFS security group from the security group drop down and un-tick the Default security group for each availability zone. Ensure that the private subnet is selected in each availability zone.
6. Click Next button then click the Next button on the Policy tab. On the final Review and Create tab click Create.

Step 4: Create PostgreSQL Database

1. Type RDS in the search bar at the top of the screen. Click the RDS title from the search results.
2. From the RDS console click the Create Database button.
3. Select the following options on the Create Database page.
 - Database Creation Method: Standard create
 - Engine Option: Aurora (PostgreSQL Compatible)
 - Hide filters: select Show versions that support Serverless v2 (if shown)
 - Available versions: select Aurora PostgreSQL (Compatible with PostgreSQL 15.4).
 - Templates: select Dev/Test
4. Scroll down and select Dev/test template. Under setting:
 - DB cluster identifier : mypostgis
 - Master username: pgadmin
 - Master Password: Enter a password of your choosing
 - Confirm master password: Re-enter your password
5. Set storage and DB instance class configuration:
 - Cluster Storage Configuration--> Configuration option: Aurora Standard
 - Instance Configuration --> DB Instance Class: Serverless V2
 - Availability & Durability --> Multi AZ Deployment : Create an Aurora Replica or Reader node in a different AZ
6. In connectivity section set the following values:
 - Compute Resource: Don't Connect to an EC2 compute resource
 - Virtual Private Cloud (VPC): Select the Geoserver-VPC you created earlier
 - VPC Security Group: Select Choose Existing and select HOLPostGIS security group that you created earlier. Leave the other options as the defaults.

7. In the Monitoring section deselect Performance Insights and Enhanced Monitoring to turn off these features. You may wish to explore these latter for your own deployments but we do not use them in this lab.
8. Expand the Additional configuration section at bottom of page and configure the following:
 - Initial Database Name: firstdb
9. You can leave remaining settings as is and click Create Database. You can close any pop up to install add-ons.

Step 5: Create AWS CloudShell environment to manage database

1. At the bottom of the AWS console browser window select CloudShell to open the AWS CloudShell command line.
2. At the top right of the CloudShell window select the Actions menu dropdown and select the Create VPC environment action.
3. Fill out the Name, VPC, Subnet and Security group settings for your instance of CloudShell to run in.
 - Name: geoserverlab.
 - Virtual Private Cloud (VPC): Make sure you select the geoserver VPC that you created earlier.
 - Subnet: Select a PRIVATE subnet.
 - Security Group: Select the HOLGeoserver security group.
4. Once dialog filled out click Create button. Make sure you use the geoserverlab CloudShell environment in latter command line steps.

Before you move on verify that your RDS database is running.

5. In RDS console check your database is available. To do this:
 - Type RDS in the top search bar then click RDS in the search results
 - From the RDS console dashboard select Databases from the left hand menu.
6. Click on the "mypostgis" name to open the database details. Click the radio button to the left of the mypostgis row where the Role column shows "Regional Cluster". In the Connectivity & Security tab click the copy icon to copy the endpoint address. Store in a text editor as we will need it to connect the database.

PART – 2: Configure Geoserver

Step 6: Install PostGIS and load data

1. If you have closed the CloudShell window; At the bottom of the AWS console browser window select CloudShell to open the AWS CloudShell command line.
2. This will open the CloudShell window.
3. Make sure you select the geoserverlab tab to use the within VPC command line environment you setup previously.
4. On the command line check your current directory and change to your home directory.

Set up PostGIS Extension:

Now the psql command line tool is installed we can connect to the database. You will need to know your database connection string which you can find in the RDS console for your database.

5. Retrieve the database end point address you copied earlier when setting up the database server. Copy and paste the following command into the terminal. Before you run it delete and past your RDS endpoint that you saved earlier. It should look something like:

```
rdsendpoint=mypostgi.cluster-abcdefghijkl.us-east-1.rds.amazonaws.com
```

```
rdsendpoint=<RDS-endpoint>
```

Then copy and paste the following command to connect to your database.

```
psql --host=$rdsendpoint \  
--port=5432 \  
--username=pgadmin \  
--password \  
--dbname=firstdb
```

This will prompt you for the pgadmin password that you entered when creating the database and then put you in the psql command line.

6. Create a separate role (user) to administer the PostGIS extension.

```
CREATE ROLE gis_admin LOGIN PASSWORD 'change_me';
```

7. Grant this role rds_superuser privileges, to allow the role to install the extension.

```
GRANT rds_superuser TO gis_admin;
```

8. Create a database to use for your PostGIS artifacts.

```
CREATE DATABASE lab_gis;
```

9. Give the gis_admin all privileges on the lab_gis database.

```
GRANT ALL PRIVILEGES ON DATABASE lab_gis TO gis_admin;
```

10. Exit the session (use quit) and reconnect to your RDS for PostgreSQL DB instance as gis_admin.

```
quit
```

```
psql --host=$rdsendpoint \  
--port=5432 \  
--username=gis_admin \  
--password \  
--dbname=lab_gis
```

11. Use CREATE EXTENSION statements to load the PostGIS extensions.

```
CREATE EXTENSION postgis;  
CREATE EXTENSION postgis_raster;  
CREATE EXTENSION fuzzystrmatch;  
CREATE EXTENSION postgis_tiger_geocoder;  
CREATE EXTENSION postgis_topology;  
CREATE EXTENSION address_standardizer_data_us;
```

12. You can verify the results by running the SQL query shown in the following example, which lists the extensions and their owners.

```
SELECT n.nspname AS "Name",  
pg_catalog.pg_get_userbyid(n.nspowner) AS "Owner"  
FROM pg_catalog.pg_namespace n  
WHERE n.nspname !~ '^pg_' AND n.nspname <> 'information_schema'  
ORDER BY 1;
```

13. Use the ALTER SCHEMA statements to transfer ownership of the schemas to the gis_admin role.

```
ALTER SCHEMA tiger OWNER TO gis_admin;  
ALTER SCHEMA tiger_data OWNER TO gis_admin;  
ALTER SCHEMA topology OWNER TO gis_admin;
```

14. You can confirm the ownership change by running the following SQL query.

```
SELECT n.nspname AS "Name",  
pg_catalog.pg_get_userbyid(n.nspowner) AS "Owner"  
FROM pg_catalog.pg_namespace n  
WHERE n.nspname !~ '^pg_' AND n.nspname <> 'information_schema'  
ORDER BY 1;
```

15. Use the following function to transfer ownership of the PostGIS objects to the

gis_admin role. Run the following statement from the psql prompt to create the function.

```
CREATE FUNCTION exec(text) returns text language plpgsql volatile AS
$$ BEGIN EXECUTE $1; RETURN $1; END; $$;
```

(Note: if scripting this you may wish to add "DROP FUNCTION IF EXISTS exec(text)" before to allow running script multiple times with same result.)

16. Next, run the following query to run the exec function that in turn runs the statements and alters the permissions.

```
SELECT exec('ALTER TABLE ' || quote_ident(s.nspname) || '.' ||
quote_ident(s.relname) || ' OWNER TO gis_admin;')
FROM (
SELECT nspname, relname
FROM pg_class c JOIN pg_namespace n ON (c.relnamespace = n.oid)
WHERE nspname in ('tiger','topology') AND
relkind IN ('r','S','v') ORDER BY relkind = 'S')
s;
```

Now we can test the PostGIS extensions are installed before we install some data.

17. To avoid needing to specify the schema name, add the tiger schema to your search path using the following command.

```
SET search_path=public,tiger;
```

18. Test the tiger schema by using the following SELECT statement.

```
SELECT address, streetname, streettypeabbrev, zip
FROM normalize_address('1 Devonshire Place, Boston, MA 02109') AS na;
```

19. Test access to the topology schema by using the following SELECT statement. This calls the createtopology function to register a new topology object with the specified spatial reference identifier (26986) and default tolerance (0.5).

```
SELECT topology.createtopology('my_new_topo',26986,0.5);
```

Load Data into PostgreSQL

Well done you have enabled PostGIS on the Amazon Aurora database and verified it is operating. Now before leaving the EC2 instance we will upload some data so we can access it from Geoserver latter.

At the top of the page is a data file that is used for the tutorial and we will download that data to load into PostGIS.

20. Quit out of the psql by typing quit and pressing enter. This will return you to the Linux shell. Type cd and hit enter to ensure you are in the home directory.

21. Enter the following command to download the file:

```
curl -o postgis-2020.zip https://s3.amazonaws.com/s3.cleverelephant.ca/postgis-2020.zip
```

22. Unzip the data using unzip.

```
unzip postgis-workshop-2020.zip
```

23. Change directory to the postgis-workshop-2020/data directory.

```
cd postgis-workshop/data
```

24. Enter the following command to restore this workshop database.

```
pg_restore --host=$rdsendpoint \  
--port=5432 \  
--username=gis_admin \  
--password \  
--dbname=lab_gis nyc_data.backup
```

Enabled PostGIS extensions and uploaded some geospatial data to the database which we will connect with from Geoserver.

Step 7: Create ECS Cluster

We will now create an ECS cluster to support Fargate container instances to host Geoserver application.

1. Type ECS into the search bar at the top of the screen and select Elastic Container Service from the search results.
2. On the ECS console select Clusters from the left hand menu then and click the Create Cluster button.
3. On the Create Cluster screen enter the following then click Create Cluster
 - Cluster Name: HOLCluster

Leave Infrastructure settings as is to support Fargate based serverless deployment. Click Create to create the cluster. This will take a few minutes to complete.

Step 8: Create ECS Task Definition

1. In the ECS console select Task Definitions in the left panel and then click the Create new Task Definition button.
2. Enter a name for the container: HOLGeoserver. You can leave the Infrastructure requirements as default.
3. Scroll down to the Container-1 section. Enter the following:
 - Name: Geoserver
 - Image URI: docker.osgeo.org/geoserver:2.26.0 - points to the public container repository. (Note: version numbers do get updated regularly. 2.25.2 also an older option)
 - Container Port: 8080 -the default Geoserver container listens on this port.
 - Port Name: geoserver-8080-tcp
4. Scroll down to Logging section. We leverage the capability here to redirect Logging information from the container to CloudWatch Logs. This will facilitate management of logs particularly as we scale out to running multiple containers in a load balanced configuration.
 - Update the awslogs-group value to /ecs/HOLgeoserver
5. Expand the Health check - optional section. Enter the following:
 - Command: CMD-SHELL, curl -f http://localhost:8080/geoserver/web || exit 1
 - Interval: 30
 - Timeout: 10
 - Start Period: 10
 - Retries: 3
6. Scroll down to **Storage - optional section**. We need to add an entry to connect to the EFS volume you created earlier. Click on the Add Volume button.

Enter the following:

- Volume type: EFS
- Volume Name: HOLDgeofsvol
- File System ID: Select the File system you create earlier. e.g HOLEFS (fs-123456789)

Click the add mount point button to add a container mount point to indicate where we mount the EFS volume in the container file system. We are going to add it at the standard location for the Geoserver data area so it can be shared across multiple container instance. Select Geoserver containers.

Enter the following:

- Container: Geoserver
 - Source Volume: HOLgeofsvol
 - Container Path: /opt/geoserver_data/
7. Scroll to bottom of the screen and click create to create the task definition for the container.

Create CloudWatch Log group

We need to verify the Cloudwatch log group was created.

1. Type **Cloudwatch** into the search bar at the top of the screen. Select the **Cloudwatch** search result.
2. Select **Log Groups** from the left hand menu.
3. Verify that the log group called **/ecs/HOLgeoserver** is listed.
4. In the Log groups area click Create Log group.
5. Name your log group with the same name used in the Task definition in last section then click create.
6. Return to the ECS console for the next steps.

Step 9: Run Geoserver

Now we have all the components in place we can run an instance of Geoserver.

1. On the task definition screen from end of last set you can Select **Deploy and Run Task**.
2. Select your ECS cluster in the **Existing Cluster** field.
3. Scroll down and open the Networking panel and expand it. Configure the following:
 - VPC: Select the **Geoserver-VPC**
 - Subnets: Make sure only the 3 subnets with **public** in their name are selected.
 - Security Group Name: Select the **HOLGeosever** security group. And remove the **default** security group
 - Public IP: make sure this is enabled.
4. Scroll to bottom and click **Create** to start the Geoserver container.

Once your task shows as running in the console click on the link in the **Task** column

to view the task details.

5. Locate the **public IP address** of your container. Copy and paste it into a text editor as we will use it shortly.

Step 10: Use and configure Geoserver

1. Open a new browser tab and enter the following :
`http://IP from last step]:8080/geoserver/web`. NOTE: URL is case sensitive!
This should open the Geoserver home page.
2. As this is the default configuration you can login with
 - **Username:** admin
 - **Password:** geoserver.

Make sure you change this when building Geoserver for your own purposes.

1. You can click on Layer Preview to view some of the built-in layers for demonstration.

Create a Workspace

We will create a Workspace for some hands on lab exercises within Geoserver to leverage the Amazon EFS file store and the Amazon Aurora PostgreSQL PostGIS database.

1. Click on Workspaces on left tab.
2. Then click Add new workspace.
3. Enter a name for the workspace (HOLworkspace) and a URI (<http://www.myholworkspace.com>), then click Save.

Now we add some data stores to illustrate accessing the connected file system and the backend database.

4. Click on the Stores link on the left panel.
5. Then click Add new Store .
6. Click on Shapefile under Vector Data Sources.
7. Enter the following details
 - **Workspace:** Select HOLworkspace
 - **Data Source Name:** HOLefsusstates

- Description: US states from EFS

Then :

- Click the browse link next to the Shapefile Location field.
 - Click the data/ link
 - Click shapefiles/ link
 - Click the states.shp link to select the States shapefile.
8. Then click Save. This takes you to the New Layer screen. Click the Publish link to publish this data source as a layer.
 9. In the Edit Layer screen update the Name to HOL EFS US States and Title to HOL EFS US States so you can identify this layer from the built-in layers.
 10. Scroll down the screen to the Bounding Boxes section. Click on the Compute from Native bounds link.
 11. Then click Save and you will see layer has been added.
 12. You can now go to the Layer Preview area and click on OpenLayers next to your new layer to see the Published layer.

Publish a layer from the PostgreSQL database.

Now let's connect to the database we created earlier and publish a layer from data in the PostgreSQL database.

1. On the Geoserver home page click on the Stores link.
2. Click Add new Store.
3. Select PostGIS under Vector Data sources.
4. Enter the following then click Save
 - Workspace: HOLworkspace.
 - Name: NYC-postgis
 - Description: postgis workshop db
 - Host: Paste in the RDS endpoint you copied earlier in the workshop.
 - Port: 5432
 - Database: lab_gis
 - user: gis_admin
 - Password: enter the password you used when creating the lab_gis user.
5. When the new layer page displays click on the Publish link next to the nyc_census_blocks row.

6. On the Edit Layer page enter a Name (HOL_nyc_census_blocks) and Description (HOL_nyc_census_blocks) to make the layer easy to locate.
7. Scroll down to the bounding boxes sections and click on the compute from native bounds link to populate bounding boxes
8. Click Save to complete layer definition.
9. Navigate to the Layer Preview page and click on the Open Layers link for this layer to preview.

Published data stored in your backend PostgreSQL PostGIS database into Geoserver.

Step 11: Create Target Group

A Target Group which will be needed when we create a Load Balancer.

A Target Group defines what resources the load balancer will send traffic to, in our case this will be the Geoserver containers. Since we need to configure a custom port number (8080) and some custom health check information we need to set this up before we dive in to create the Load Balancer.

1. Open the EC2 console by typing EC2 into the search bar at the top of the screen. Click the EC2 search result.
2. In the left hand menu locate the Load Balancing section and click Target Groups
3. Click the Create Target Group button.
4. On the Specify group details page enter the following:
 - Choose Target Type: IP addresses
 - Target Group Name: HOLGEOTG
 - Protocol: HTTP
 - Port :8080
 - VPC: Select the Geoserver-vpc
4. Scroll down to the Health Checks area and expand the advanced health check settings. Enter the following:
 - Health Check Path: /geoserver/web
 - Success Codes: 200,302
5. Click Next and then on next screen click Create Target Group.

6. Select the newly created Target Group and in Actions menu select Edit Target Group Attributes.
7. In the Target selection configuration Enable stickiness and then click Save changes. This is needed as Geoserver needs sticky sessions to operate behind a load balancer.

Step 12: Create Load Balancer

An Application Load Balancer (ALB) is used when we scale out to multiple Geoserver tasks running as a service.

1. In the left hand menu locate the Load Balancing section and click Load Balancers
2. Click Create load balancer.
3. Click Create under Application Load Balancer.
4. Enter a Name (HOLALB) for the load balancer.
5. In the network mapping section select the geoserver-vpc that you created and select all 3 availability zones. Ensure that the public subnet in each availability zone is selected.
6. In the Security Groups section select the security group called HOLALB. If the default security group is preselected then untick it to remove it.
7. In the Listener and routing section leave Listener set to port 80. In the Default action drop down select the Target group you created in last step: HOLGEOTG in dropdown.
8. Click Create Load Balancer at bottom to create the load balancer.

Step 13: Create Load Balanced Geoserver Service

At present we have a single instance of a Geoserver container running on Fargate. To complete the workshop, we will now create a service to run multiple container tasks and expose using a load balancer.

1. Type ECS into the search bar at the top of the screen and select Elastic Container Service from the search results.
2. On the ECS cluster list page click the name of your cluster (HOLCluster). Click the Services tab, then click on Create button

3. Scroll down to the Deployment Configuration section. Enter the following:
 - Family: Select the HOLgeoserver task definition
 - Service Name: HOLgeoserverservice
 - Desired tasks: 2
4. Scroll down to the Networking section and expand it. Enter the following:
 - VPC: select the Geoserver-vpc
 - Subnets: Ensure only the public subnets are selected
 - Security Group Name: Select the HOLgeoserver security group. If the default security group is selected then de-select it.
5. Expand the Load balancing section. Enter the following:
 - Load Balancer Type: Application Load Balancer
 - Application Load Balancer: select Use an existing Load Balancer
 - Load Balancer: select HOLALB
 - Listener: Select Use an existing listener then select 80:HTTP from the drop down.
 - Target Group: Select Use an existing Target Group then select HOLGEOTG from the target group name drop down.
 - Health Check Grace Period: 300
6. Scroll down to the Service Auto scaling area. Select Use service auto scaling and enter the following:
 - Minium Number of tasks: 2
 - Maximum Number of Tasks: 9
 - Policy Name: HOLscalepolicy
 - ECS Service Metric: ECSServiceAverageCPUUtilization
 - Target Value: 70
 - Scale-Out cooldown period: 300
 - Scale-in cooldown period: 300
7. Scroll to bottom and click Create.
8. The deployment will take a few minutes and you can optionally click on the view in.
9. CloudFormation button to see what resources are being built out.
10. To monitor the deployment click on the service name to see the service details, including deployment status.
11. Once the service is running and targets are healthy click on View load balancer button to open the Load Balancer console.

12. In the Load Balancer details copy the DNS name of the Load Balancer.
13. To test the deployment and view Geoserver open a new browser window.
14. Paste in the load balancer DNS URL and postpend /geoserver/web to the end.

Geoserver should come up and you can log in with the same username/password as before and view the layers you created previously.

Conclusion:

In conclusion, this project demonstrates the power of AWS in transforming traditional server management into a seamless, automated process. This project serves as a valuable reference for organizations looking to implement similar architectures, showcasing best practices and the potential for future scalability in cloud-based geospatial solutions. Automated Geoserver deployment running on Fargate containers without having to manage your own server infrastructure.

Note: Delete all the resources at the end to avoid incurring costs.